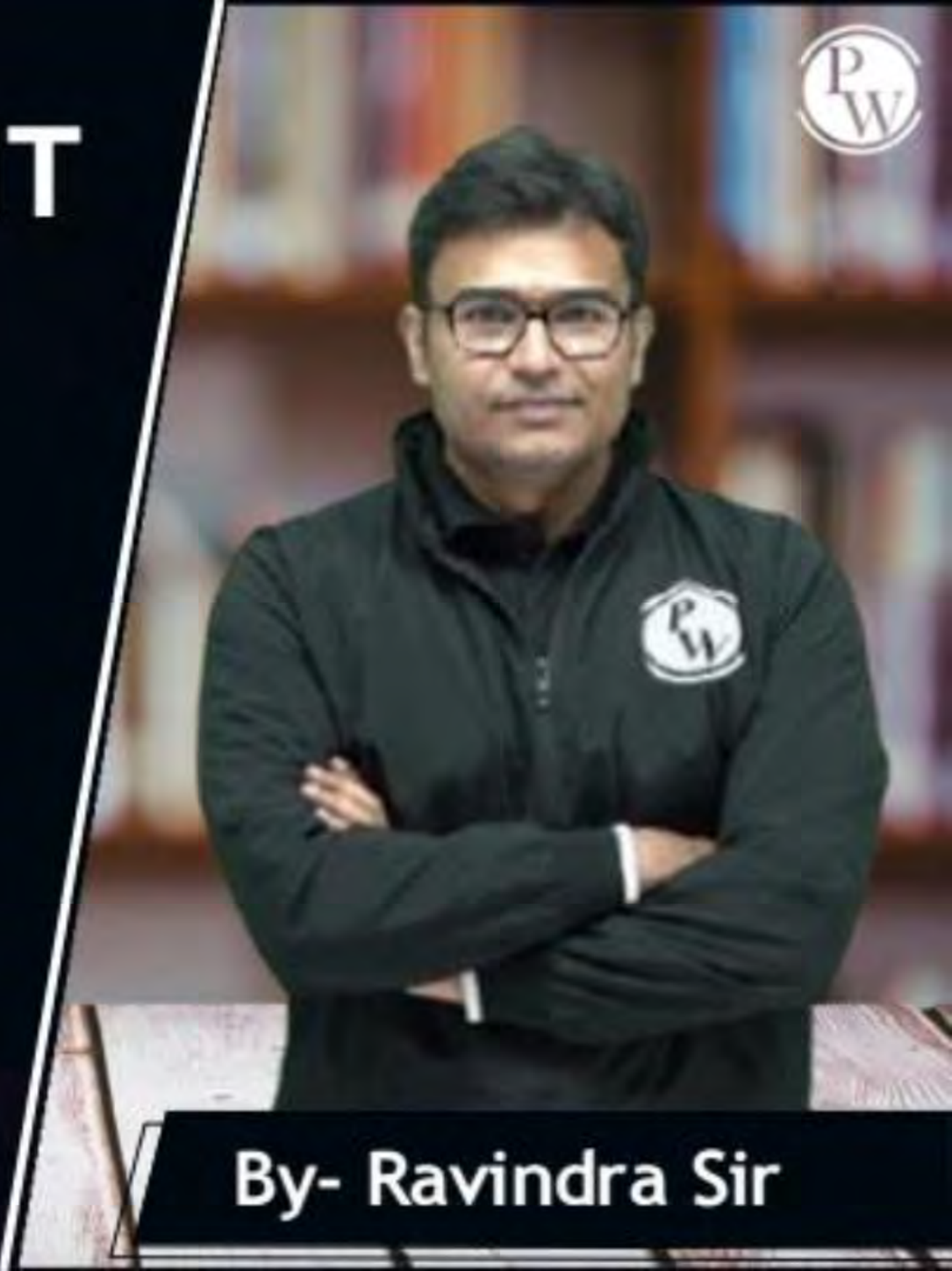
Computer Science & IT

COMPILER DESIGN
(CD)

Parsers

Lecture No. 15



By- Ravindra Sir

Recap of Previous Lecture



- Topic
- Topic
- Topic

Parser

Topics to be Covered



- Topic
- Topic
- Topic

Parsers :

5 Rules of RBR

1) Younger → Blessing
2) Elder → Respect
3) Poor → help
4) Talent → help
5) Humble

Join Telegram

@RBRSIRPW





Inspiring Stories : Bholanath Chatterjee

Bholanath Chatterjee

Background: Born in Bengal in the late 1800s. Joined the freedom movement.

Struggle: Arrested in 1916 and beaten badly in Yerawada Jail.

Education: Learned from Bengal's freedom groups.

Achievement: Never told the British anything, no matter how much they tortured him. Official reports say suicide, but many believe he chose death over betraying his friends.

Impact: Became a symbol of loyalty—showing that silence can also be a weapon.



Inspiring Stories : Gulab Kaur

Gulab Kaur (Ghadar woman)

Background: Born ~1890 in Bakshiwala, Punjab. Left home and went overseas.

Struggle: Joined the Ghadar crowd abroad where Indians planned to hit the Raj hard. Life was risky and lonely.

Education: No big school — learned by doing, organizing, printing papers.

Achievement: Traveled back to India with other Ghadarites, distributed rebel pamphlets, helped run secret printing and mobilization in Punjab.

Impact: A frontline woman in a movement most people only hear about in pieces — showed that the fight was global and women led the front too.



Inspiring Stories : Gour Hari Das

Gour Hari Das (the man who kept fighting after freedom)

Background: From Odisha; arrested as a teen for hoisting the flag.

Struggle: Did time as a schoolboy; later spent 30 years fighting the government to get official recognition for his freedom work.

Education: Grew up in a freedom-family; learned protest from elders.

Achievement: Hoisted the flag at 14, jailed for that — then spent decades reopening files, filing petitions, writing thousands of letters to prove he'd helped the movement.

Impact: His second fight (for recognition) is its own lesson — persistence after work is finished. If he can slow-grind the system for 30 years, you can grind the GATE syllabus.



Inspiring Stories : Bonangi Pandu Padal

Bonangi Pandu Padal (tribal striker from Visakhapatnam)

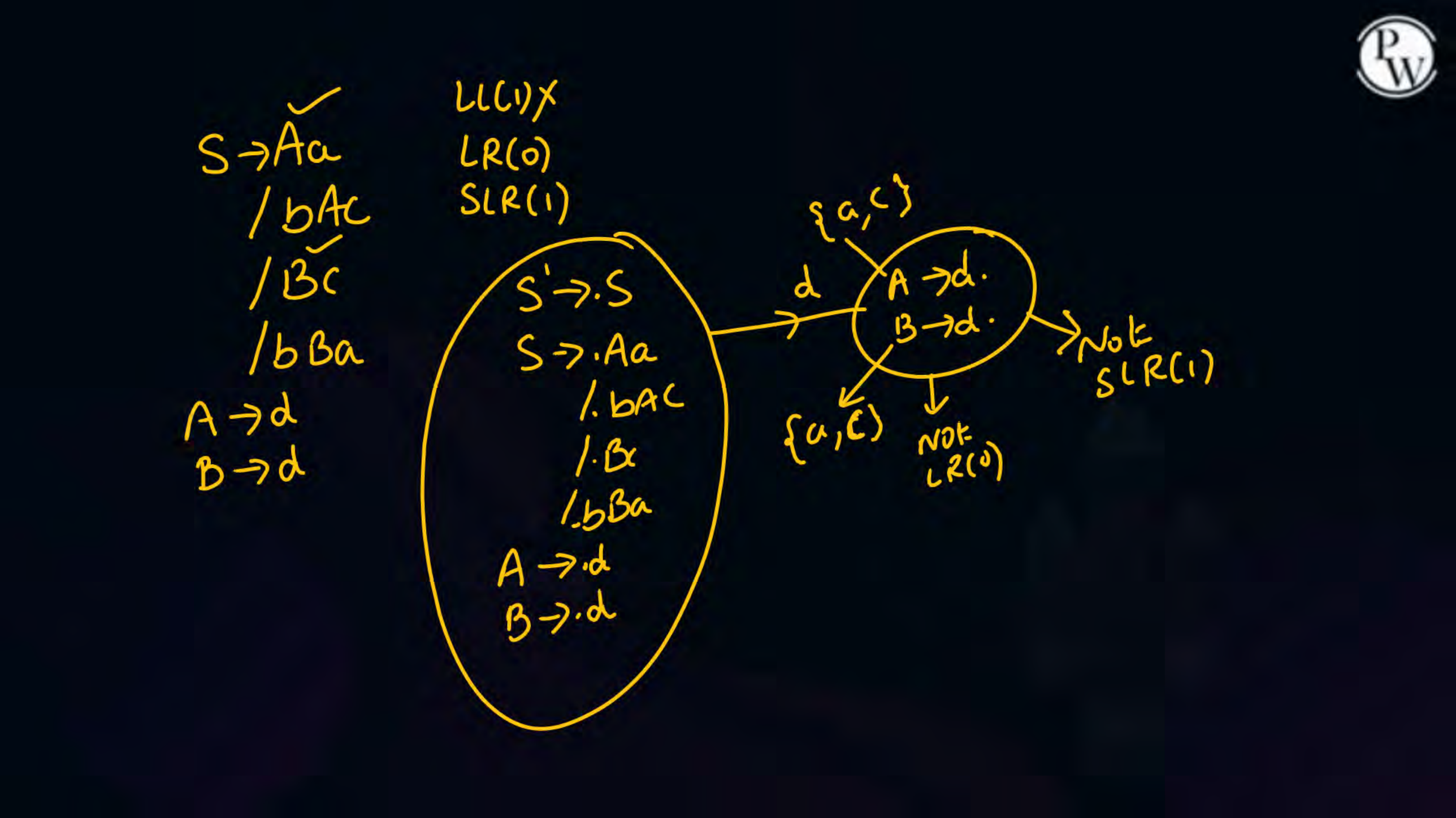
Background: Born 1890 in a tribal village in the Godavari/ Visakhapatnam area.

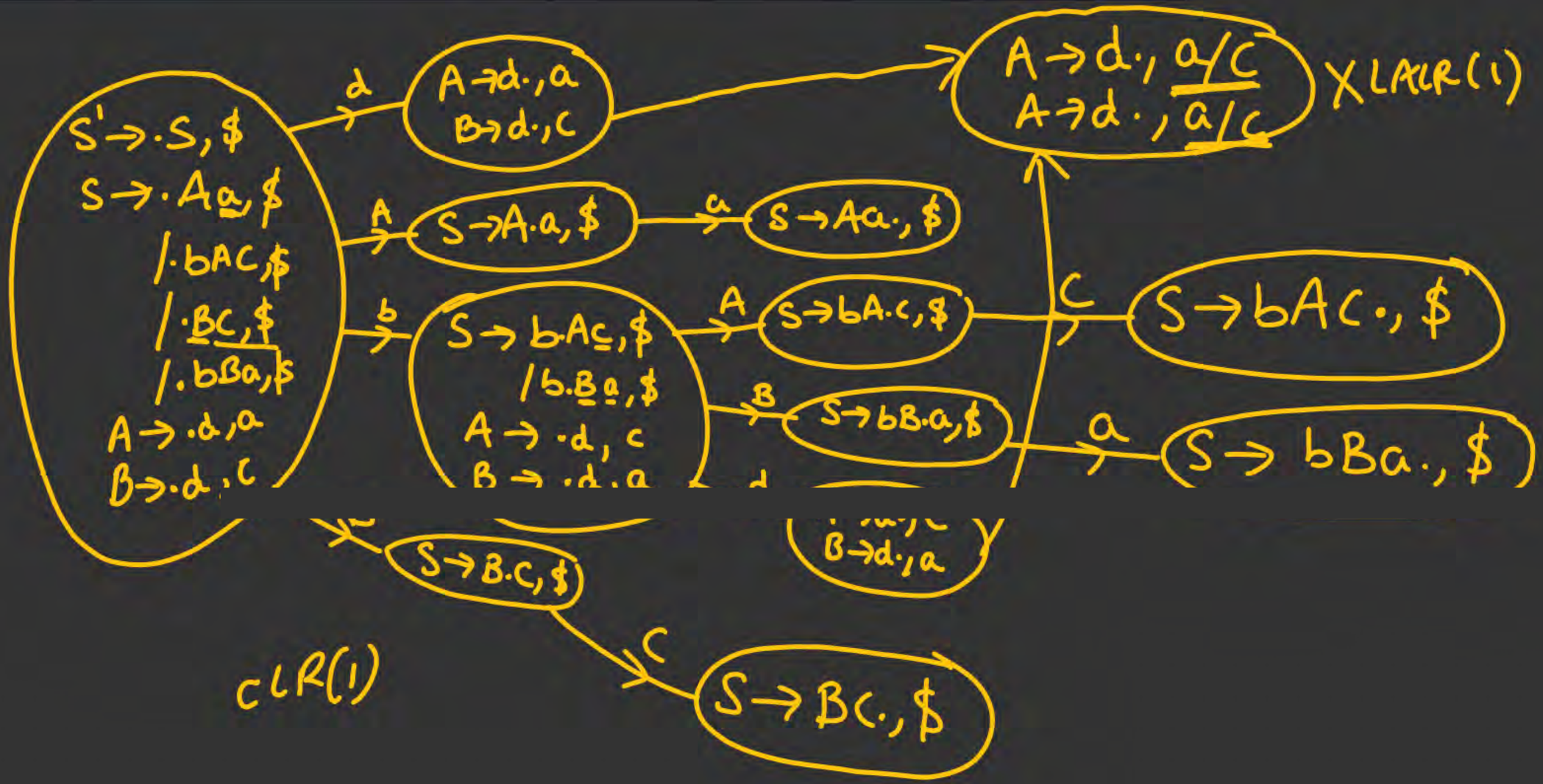
Struggle: Lived under brutal forest laws and colonial raids on tribal land.

Education: Tribal learning — survival, tracking, community leadership.

Achievement: Joined local armed actions (with Alluri-type groups), attacked police posts that enforced harsh forest rules, got jailed.

Impact: Became a local legend — proof that freedom came from villages and forests, not just cities..


S → Aa
/ bAc
/ Bc
/ bBa
A → d
B → d
LL(1) X
LR(0)
SLR(1)
S' → .S
S → .Aa
/ .bAc
/ .Bc
/ .bBa
A → .d
B → .d
d
{a, c}
A → d.
B → d.
{a, $}
NOT LR(0)
NOT SLR(1)


S'→.S,$
S→.Aa,$
/.bAC,$
/.BC,$
/.bBa,$
A→.d,a
B→.d,c
d
A→d.,a
B→d.,c
A→d.,a/c
A→d.,a/c
X LALR(1)
A
S→A.a,$
a
S→Aa.,$
b
S→bAc,$
/b.Ba,$
A→.d,c
B→.d,a
A
S→bA.c,$
c
S→bAC.,$
B
S→bB.a,$
a
S→bBa.,$
B→d.,a
S→B.C,$
C
S→BC.,$
CLR(1)

$S \to A$

$A \to AB/\epsilon$

$B \to aB/b$

$S' \to .S, \$$

$S \to .A, \$$

$A \to .AB, \$$

$/. \quad , \$$

$A \to .AB, a/b$

$/. \quad , a/b$

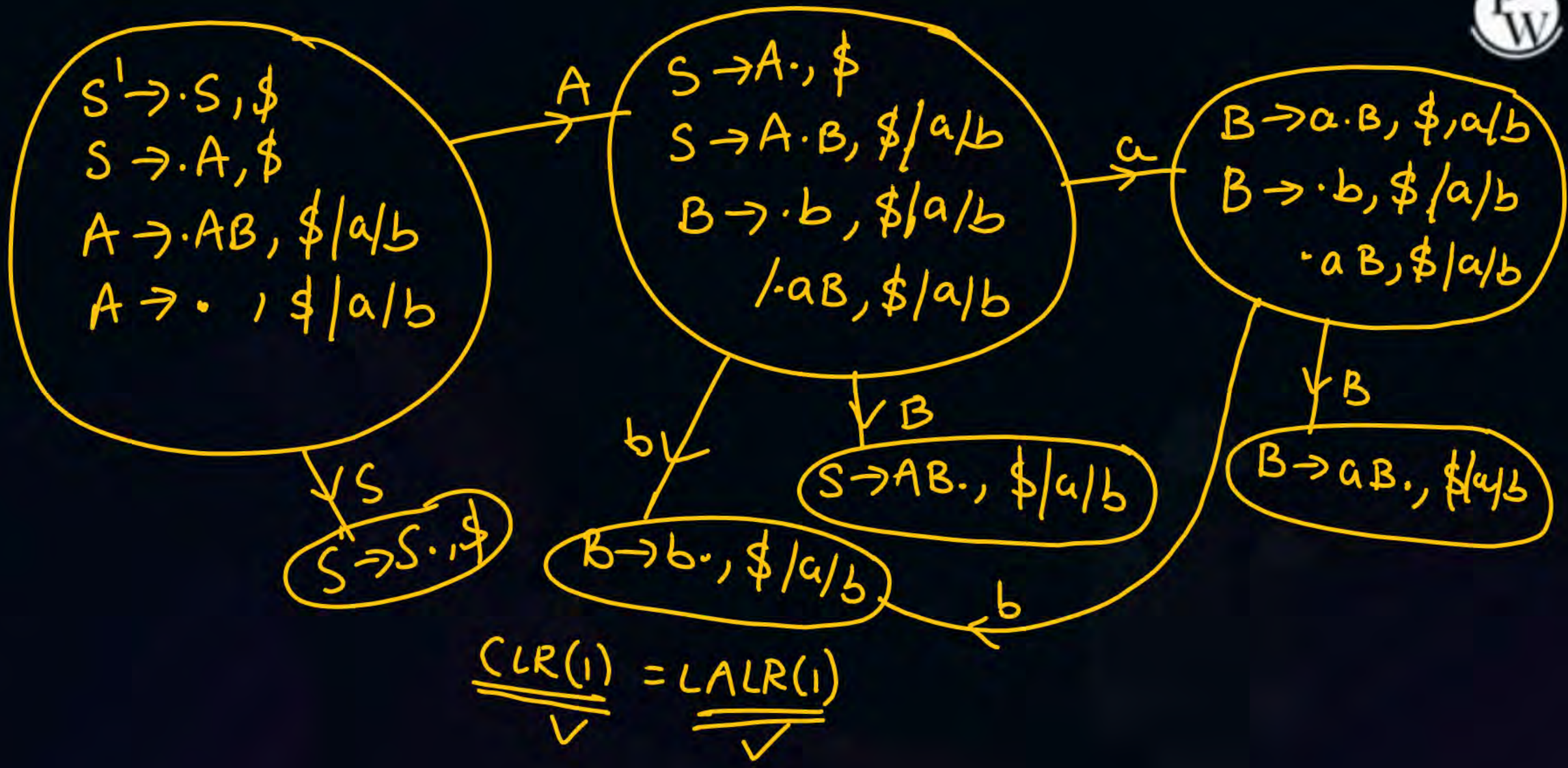
$\Rightarrow$

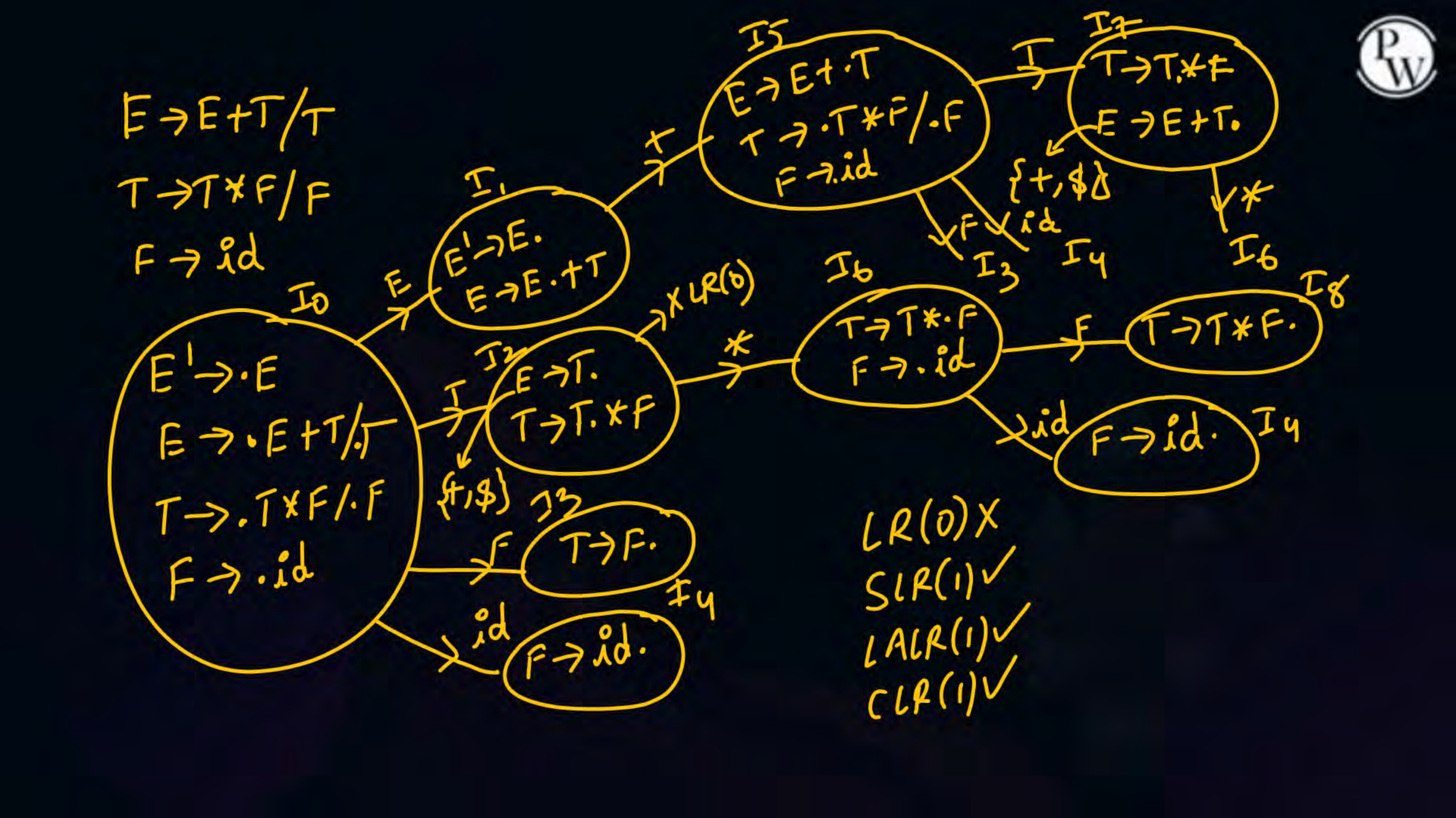
$S' \to .S, \$$

$S \to .A, \$$

$A \to .AB, \$/a/b$

$/. \quad , \$/ab$


S'→·S, $
S→·A, $
A→·AB, $/a/b
A→· , $/a/b
A
S→A·, $
S→A·B, $/a/b
B→·b, $/a/b
B→·aB, $/a/b
a
B→a·B, $/a/b
B→·b, $/a/b
B→·aB, $/a/b
S
S→S·, $
b
B
S→AB·, $/a/b
B→b·, $/a/b
b
B
B→aB·, $/a/b
CLR(1) = LALR(1)


E→E+T/T
T→T*F/F
F→id
I0
E'→·E
E→·E+T/·T
T→·T*F/·F
F→·id
E
I1
E'→E.
E→E.+T
+
I5
E→E+·T
T→·T*F/·F
F→·id
T
I7
T→T.*F
E→E+T.
{+,$}
*
I6
F
I3
id
I4
I2
E→T.
T→T.*F
I
{*,+,$}
X LR(0)
*
I6
T→T*·F
F→·id
F
I8
T→T*F.
id
F→id.
I4
{*,+,$}
I3
T→F.
F
I4
id
F→id.
LR(0) X
SLR(1) ✓
LALR(1) ✓
CLR(1) ✓

Gate:

→ Consider SLR(1), and LALR(1) tables for a CFG

✗ a) Goto of both tables may be different

✓ b) Shift entries are identical in both tables

✓ c) Reduce entries in tables may be different

✓ d) Error entries in tables may be different

$\rightarrow n_1 = |SLR(1))|$ $n_2 = |LALR(1)|$ $n_3 = |CLR(1)|$

$n_1 = n_2 \leq n_3$

→ S → C C
C → c C / d

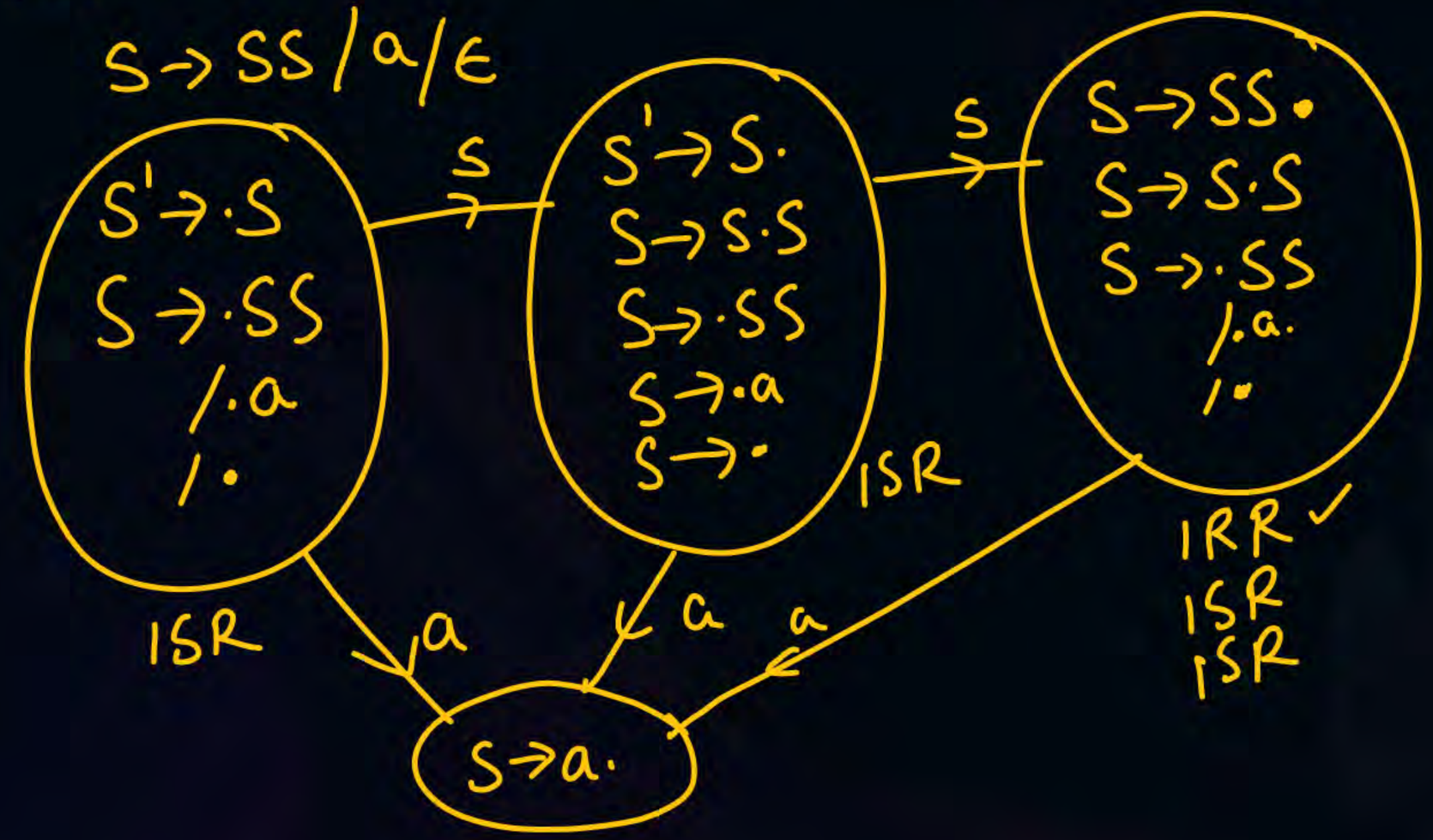
a) LL(1)

b) SLR(1) but not LL(1)

c) LALR(1) but not SLR(1)

d) CLR(1) but not LALR(1)

Find the no of SR and RR conflicts in dfa with LR(0) items

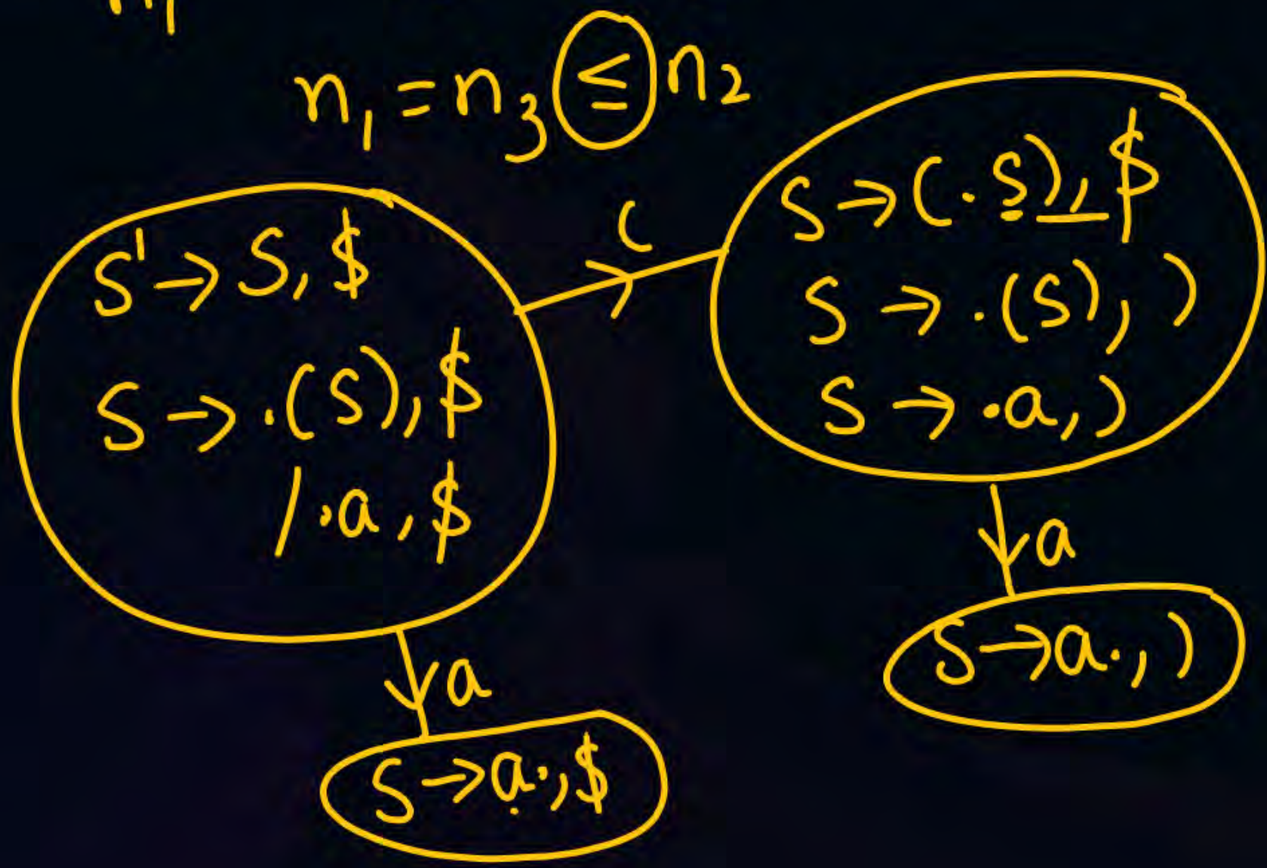


→ S → (S)
/a

SLR(1) n_1 CLR(1) n_2 LALR(1) n_3

$n_1 = n_3 < n_2$

$n_1 = n_3 \leq n_2$




unambiguous gram
CLR(1)
LALR(1)
SLR(1)
LR(0)
LL(1)
opp
ambiguous

Comparison of parsers : TDP Vs BUP :-

Out of all TDP's, most widely used one is LL(1) parser

Out of all BUP's, most widely used one is LALR(1) parser

Design :- TDP is simple to construct than BUP

Table Size :- LALR(1) parser table is exponential to the size of grammar

This is not the case with LL(1). LL(1) table size is bounded by square of the size of grammar. The size of BUP is double the size of TDP

Pow :- $LL(K) \subset LR(K)$

Error detection:- Predictive nature of TDP's allow errors to be detected at the earliest possible time. The TDP parser stack can be used to repair the i/p as it contains information on what is expected to be seen in constrast to the BUP stack which stack contains information about what has already been encountered in parsing

$S \to (S)/\epsilon$

	(	)	$
S	S→(S)	S→ϵ	→ϵ

→ LL(1) parser is simple TDP that can parse only a small class of grammars as there is restriction on grammar that it should be free of LR and should be left factored.

→ LR(0) is simplest of all LR parsers but is not used practically as it is less powerful. It doesn't use lookahead in the making of parsing decisions, hence can be used to parse only a small class of grammar.

SLR(1) is better than LR(0) as it uses lookahead

SLR(1) is not powerful like CLR(1).

CLR(1) is the most powerful parser but requires more table space

So LALR(1) is preferred as its table size is less than CLR(1) and uses LR(1) items

	LL(1)	LR(0)	SLR(1)	LALR(1)	CLR(1)
1) S → a	✓	✓	✓	✓	✓
2) E → E+T/T T → a	X	✓	✓	✓	✓
3) E → E+T/T T → T*F/F F → id	X	X	✓	✓	✓
4) S → Aa/bAc/dc/bda A → d	X	X	X	✓	✓
5) S → Aa/bAc/Bc/bBa A → d, B → d	X	X	X	X	✓
6) S → a/A A → a	X	X	X	X	X

Error recovery in LR parsers :-

An LR parser will detect error when it consults the parsing table and find a blank entry. For each blank entry, there will be a pointer to error recovery function. So that whenever the blank entry is reffered, the corresponding subroutine is called. For example e_1, e_2, e_3 and e_4 are pointers to the error recovery functions

	id	+	*	(	)	$
0	S_3	e_1	e_2	S_2	e_3	e_4

e_1 tells that an operand is missing at the beginning of the expression. So push id and issue end message. "missing operand"

e_3 tells that expression is starting at). So end recovery here can be to delete i/p symbol and issue end message.

Parser Construction with ambiguous grammars :-

$E \rightarrow E+E$
$\quad / E*E$
$\quad / id$
ambi

$E \rightarrow E+T / T$
$T \rightarrow T*F / F$
$F \rightarrow id$
unamb

Advantages of ambiguous grammars :-

→ Ambiguous grammars are shorter and natural
→ If a parser is constructed with ambiguous grammar, we can change the associativity and precedence at a later point of time

adv+

No wastage of time

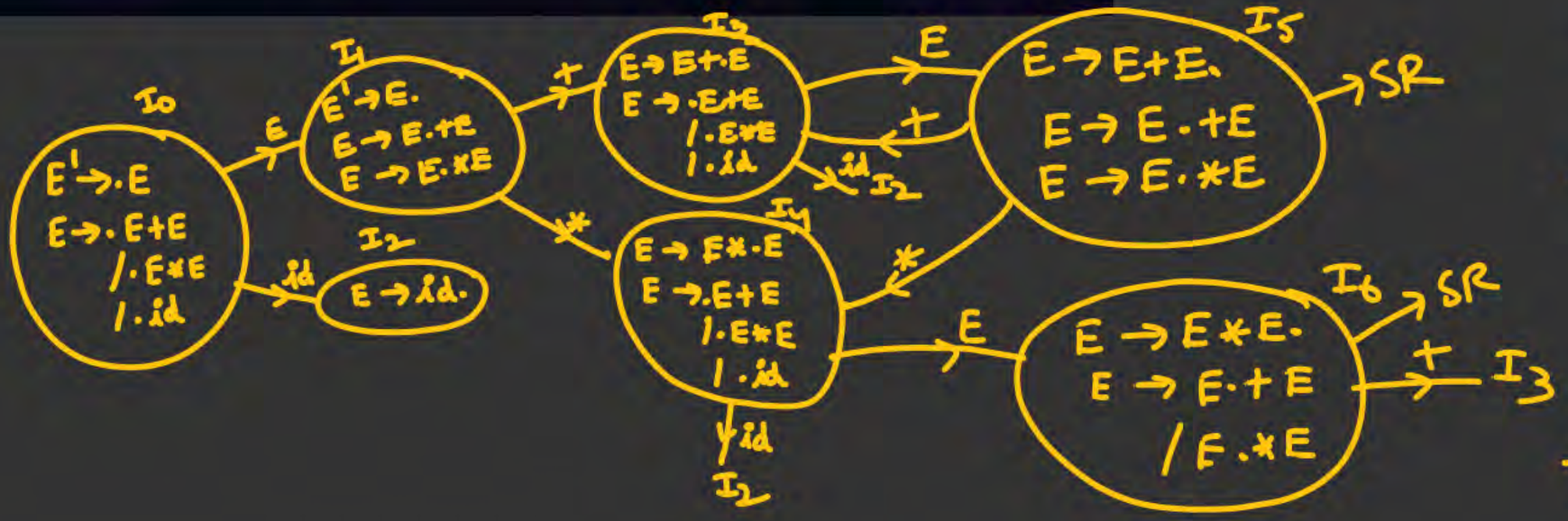
E
I
id

E
I
T
I
F
I
id

Let us construct SLR(1) parsing table for ambiguous grammar

$$E \to E + E \mid E * E \mid id$$

→ we will get conflicts, if we can resolve the conflicts we can use the parsing table



TOC - YT

CD - paid

DBMS - Sat ✓

algo - mondy

CN - paid

I_5 $E \to E+E.$ / $E \to E.+E$ — R / S → R

I_5 $E \to E+E.$ / $E \to E.*E$ — R / S → S

I_6: $E \to E*E.$ / $E \to E.+E$ — R / S → R

I_6: $E \to E*E.$ / $E \to E.*E$ — R / S → R

THANK - YOU